Projeto Prático de Bases de Dados

UC de Bases de Dados, Engenharia Informática, UFP, 3º ano, 1º semestre

1. Objetivos

O projeto tem como objetivo desenvolver uma base de dados relacional a partir de um **problema do mundo real**, percorrendo as principais etapas de análise, modelação, implementação e validação.

O trabalho será realizado em **grupos de 2 estudantes** e desenvolvido incrementalmente durante o semestre.

O processo global é:

Problema real → Requisitos → Modelo conceptual → Modelo relacional → SQL/PostgreSQL → Dados e consultas → Testes

Pretende-se que os estudantes sejam capazes de:

- analisar e delimitar um problema;
- identificar requisitos, regras de negócio, atores e necessidades de acesso;
- construir um modelo conceptual Entidade–Relacionamento;
- transformar o modelo conceptual num modelo relacional;
- analisar a normalização do modelo;
- implementar a base de dados em PostgreSQL;
- definir e testar mecanismos de integridade;
- povoar e consultar a base de dados;
- utilizar vistas, papéis e privilégios para controlar o acesso aos dados;
- utilizar transações e analisar situações de concorrência;
- analisar o desempenho de consultas e a utilização de índices;
- documentar e versionar o processo de desenvolvimento.

2. Contexto do problema

O grupo deverá desenvolver o projeto para **um dos contextos disponibilizados pelo docente**.

O contexto descreve o domínio geral, mas não especifica necessariamente todos os requisitos e regras. Faz parte do trabalho analisar o problema, definir o âmbito e tomar decisões devidamente justificadas.

Contexto A — Alojamento local / Hotel

Pretende-se gerir informação associada ao funcionamento de uma unidade de alojamento, incluindo, entre outros conceitos relevantes, quartos, tipos de quarto, clientes, reservas, pagamentos e serviços.

O sistema deverá permitir representar adequadamente aspetos como disponibilidade, períodos de reserva, ocupação e utilização de serviços.

Contexto B — Transportes / Viagens

Pretende-se gerir informação associada a um sistema de transportes de passageiros, incluindo, entre outros conceitos relevantes, viagens, veículos, motoristas, rotas, paragens, passageiros e bilhetes.

O sistema deverá permitir representar adequadamente aspetos como horários, lugares disponíveis, viagens e emissão/utilização de bilhetes.

Contexto C — Loja online

Pretende-se gerir informação associada a uma loja online, incluindo, entre outros conceitos relevantes, clientes, produtos, categorias, encomendas, linhas de encomenda, pagamentos e stock.

O sistema deverá permitir representar adequadamente a realização de encomendas, disponibilidade de produtos, pagamentos e atualização de stock.

Os conceitos apresentados são **indicativos**. A identificação e organização dos conceitos relevantes fazem parte do trabalho de análise e modelação.

3. Organização e cronograma

O projeto é constituído por **três fases incrementais**, desenvolvidas no mesmo repositório GitHub.

Fase	Trabalho principal	Entrega
Fase 1	Análise e modelação conceptual	**Semana 5 — v1.0**
Fase 2	Projeto lógico e implementação	**Semana 10 — v2.0**
Fase 3	Exploração, controlo de acesso e testes	**Final do semestre — v3.0**

As fases **não são projetos independentes**. Cada nova fase deverá fazer evoluir o trabalho anterior, corrigindo e melhorando o projeto quando necessário.

Cada entrega deverá corresponder a uma **release/tag Git**:

- `v1.0` — Fase 1;
- `v2.0` — Fase 2;
- `v3.0` — versão final.

Nas entregas `v1.0` e `v2.0` será fornecido **feedback qualitativo**. Não será atribuída uma classificação quantitativa autónoma a estas entregas.

A **classificação quantitativa será atribuída apenas no final do semestre**, considerando a versão final e, quando relevante, a evolução do projeto ao longo das três fases.

4. GitHub e documentação

Cada grupo deverá utilizar um **único repositório GitHub** para todo o projeto.

O grupo tem liberdade para organizar o repositório, desde que a estrutura seja clara e permita localizar facilmente a documentação, modelos, scripts SQL, dados e testes.

A documentação deverá utilizar predominantemente **Markdown**, podendo incluir diagramas, tabelas, imagens e outros elementos relevantes.

O `README.md` deverá constituir o ponto de entrada do projeto e identificar, pelo menos:

- contexto escolhido;
- elementos do grupo;
- objetivo e âmbito;
- organização geral do trabalho;
- instruções necessárias para reproduzir a base de dados e os testes.

O desenvolvimento deverá ser realizado de forma incremental e ficar registado através de Git.

Os commits deverão possuir mensagens adequadas e ser realizados ao longo do semestre. O número de commits, por si só, não constitui uma medida da qualidade ou da contribuição individual.

5. Fase 1 — Análise e modelação conceptual

Entrega: Semana 5 — `**v1.0**`

A primeira fase corresponde à análise do problema e à construção do modelo conceptual.

Nesta fase não deverá ser realizada a implementação da base de dados nem deverão ser tomadas decisões específicas de PostgreSQL.

5.1. Análise e âmbito

O grupo deverá analisar o contexto e delimitar claramente o problema considerado.

Deverá ser elaborado um pequeno **glossário**, identificando e clarificando os principais conceitos do domínio.

5.2. Requisitos de dados

Deverão ser identificados e numerados os principais requisitos relativos à informação que a base de dados deverá representar.

Por exemplo:

```
RD01 – ...
RD02 – ...
RD03 – ...
```

Os requisitos deverão descrever **que informação é necessária**, sem antecipar tabelas ou outras decisões de implementação.

5.3. Regras de negócio

Deverão ser identificadas e numeradas as principais regras que condicionam os dados e as relações entre eles.

Por exemplo:

```
RN01 – ...
RN02 – ...
RN03 – ...
```

As regras deverão resultar do problema real e ser suficientemente precisas para que posteriormente seja possível determinar como serão garantidas pela base de dados.

Não deverão ser inventadas regras apenas para justificar a utilização de determinado mecanismo SQL.

5.4. Atores e necessidades de acesso

Deverão ser identificados os principais **atores ou papéis do domínio** que necessitam de interagir com a informação.

Para cada um deverão ser descritas, de forma conceptual, as principais necessidades de acesso.

Por exemplo:

Ator/papel	Necessidades de acesso
Operador	consultar e registar ...
Gestor	consultar ... e alterar ...
Auditor	consultar ... sem alterar ...

Nesta fase deverão ser descritas necessidades do mundo real, sem utilizar ainda `ROLE`, `GRANT` ou outros mecanismos específicos do PostgreSQL.

5.5. Necessidades de informação

O grupo deverá identificar perguntas relevantes às quais a futura base de dados deverá ser capaz de responder.

Por exemplo:

```
Q01 – ...
Q02 – ...
Q03 – ...
```

Estas questões serão posteriormente concretizadas através de consultas SQL.

Nesta fase não deverão ser escritas as consultas SQL correspondentes.

5.6. Modelo conceptual

Deverá ser elaborado um modelo conceptual **Entidade–Relacionamento (ER)**.

A notação de referência é a **notação de Chen**, podendo ser utilizada outra notação quando previamente aceite pelo docente.

O modelo deverá representar corretamente, quando aplicável:

- entidades;
- atributos;
- identificadores;
- relacionamentos;
- cardinalidades;
- participação;
- atributos de relacionamentos;
- entidades fracas;
- outros elementos relevantes.

Recomenda-se a utilização de **diagrams.net/draw.io**, podendo ser utilizadas outras ferramentas adequadas.

Deverá ser disponibilizada uma representação visual do modelo e, sempre que possível, o respetivo ficheiro editável.

O modelo deverá ser coerente com os requisitos, regras de negócio e necessidades de informação identificados.

Nesta fase deverá ser mantido o nível conceptual, evitando elementos próprios do modelo relacional ou da implementação SQL.

6. Fase 2 — Projeto lógico e implementação

Entrega: Semana 10 — `**v2.0**`

A segunda fase transforma o modelo conceptual numa base de dados relacional implementada em PostgreSQL.

Os elementos da Fase 1 deverão ser corrigidos ou melhorados sempre que necessário.

6.1. Modelo relacional

O modelo conceptual deverá ser transformado num modelo relacional.

Deverão ser identificados:

- relações e atributos;
- domínios relevantes;
- chaves candidatas;
- chaves primárias;
- chaves estrangeiras;
- restrições de integridade.

As principais decisões do mapeamento **ER → modelo relacional** deverão ser justificadas.

6.2. Normalização

O grupo deverá analisar a normalização do modelo relacional, identificando as dependências funcionais relevantes.

Deverão ser consideradas, pelo menos:

- primeira forma normal — 1NF;
- segunda forma normal — 2NF;
- terceira forma normal — 3NF.

As decomposições efetuadas deverão ser justificadas.

Não deverão ser realizadas decomposições artificiais apenas para demonstrar o processo de normalização.

6.3. Implementação PostgreSQL

O modelo deverá ser implementado em **PostgreSQL**, utilizando scripts SQL versionados no repositório.

Deverão ser utilizados adequadamente, quando aplicáveis:

- `CREATE TABLE`;
- `PRIMARY KEY`;
- `FOREIGN KEY`;
- `NOT NULL`;
- `UNIQUE`;
- `CHECK`;
- tipos de dados apropriados;
- outras restrições declarativas.

Sempre que uma regra de negócio possa ser garantida adequadamente através de uma restrição declarativa, esta deverá ser preferida a mecanismos mais complexos.

Os scripts deverão permitir recriar a base de dados de forma previsível.

6.4. Dados

A base de dados deverá ser povoada com dados suficientemente diversificados para permitir a sua exploração e teste.

Os dados deverão representar situações normais e casos relevantes associados às regras de negócio.

Poderão posteriormente ser utilizados conjuntos de dados sintéticos de maior dimensão para os testes de desempenho.

6.5. Início das consultas SQL

Deverá ser iniciada a implementação das necessidades de informação `Q01`, `Q02`, ... identificadas na Fase 1.

As consultas deverão utilizar os mecanismos SQL estudados até ao momento.

A avaliação quantitativa das consultas será realizada sobre o conjunto final apresentado em `v3.0`.

7. Fase 3 — Exploração, controlo de acesso e testes

Entrega: Final do semestre — `**v3.0**`

A terceira fase corresponde à conclusão e validação experimental do projeto.

A versão final deverá integrar as correções e melhorias realizadas sobre as fases anteriores.

7.1. Consultas SQL

O conjunto final de consultas deverá responder às principais necessidades de informação identificadas na Fase 1.

Deverão existir consultas com diferentes graus de complexidade e utilizar, quando adequados:

- seleção, projeção e ordenação;
- `JOIN` e múltiplos `JOIN`;
- funções de agregação;
- `GROUP BY`;
- `HAVING`;
- subconsultas;
- `EXISTS` / `NOT EXISTS`;
- outras construções SQL estudadas na UC.

Poderão ser utilizadas outras construções, como CTE (`WITH`), quando forem adequadas ao problema.

Não deverão ser criadas consultas artificiais apenas para demonstrar determinada construção SQL.

7.2. Vistas, papéis e controlo de acesso

O grupo deverá definir e justificar **pelo menos uma vista (** `**VIEW**` **)** relevante para o domínio.

Deverão também ser concretizadas em PostgreSQL as necessidades de acesso identificadas na Fase 1.

O grupo deverá:

- definir pelo menos **dois papéis (** `**ROLE**` **) com necessidades de acesso distintas**;
- atribuir os privilégios adequados utilizando `GRANT` e, quando relevante, `REVOKE`;
- aplicar o **princípio do menor privilégio**, atribuindo apenas os acessos necessários;

- utilizar vistas quando estas forem adequadas para disponibilizar acesso controlado à informação;
- testar operações que devam ser permitidas e operações que devam ser recusadas.

Deverá existir uma relação clara entre:

ator/papel do domínio → necessidade de acesso → ROLE PostgreSQL → privilégios

7.3. Integridade e triggers

O grupo deverá analisar as regras de negócio e distinguir entre:

1. regras garantidas através da estrutura e das restrições declarativas (`PRIMARY KEY` , `FOREIGN KEY` , `UNIQUE` , `NOT NULL` , `CHECK`);
2. regras que exigem mecanismos adicionais.

Deverá ser implementado e testado **pelo menos um trigger**, associado a uma regra de negócio relevante que não seja adequadamente garantida através das restrições declarativas elementares.

A utilização do trigger deverá ser justificada.

Não deverá ser utilizado um trigger quando uma restrição declarativa mais simples e adequada for suficiente.

7.4. Testes de integridade e controlo de acesso

Deverão ser realizados testes positivos e negativos aos principais mecanismos implementados.

Os testes deverão incluir situações como:

- dados válidos;
- violação de chaves ou valores `UNIQUE` ;
- chave estrangeira inexistente;
- `NULL` não permitido;
- violação de `CHECK` ;
- ativação e comportamento do trigger;
- operação autorizada para determinado papel;
- tentativa de operação não autorizada.

Deverá ser possível estabelecer relações como:

regra de negócio → mecanismo de integridade → teste

e:

necessidade de acesso → privilégio → teste

Cada teste deverá identificar o resultado esperado e o resultado observado.

7.5. Transações

Deverão ser realizadas experiências que permitam observar propriedades transacionais do PostgreSQL.

Deverão ser utilizados, quando adequados:

- `BEGIN`;
- `COMMIT`;
- `ROLLBACK`;
- operações relacionadas executadas como uma unidade;
- situações de falha e respetivo efeito sobre os dados.

As experiências deverão permitir observar e discutir, em particular, a **atomicidade** das operações.

Deverão ser documentados o estado inicial, operações realizadas, resultado esperado, resultado observado, estado final e conclusão.

7.6. Concorrência e isolamento

Deverão ser realizadas experiências utilizando **duas ou mais sessões PostgreSQL independentes**.

Deverão ser analisadas situações relevantes para o contexto do projeto, podendo incluir:

- leitura e atualização concorrente;
- atualização concorrente da mesma informação;
- bloqueios;
- comportamento de transações concorrentes;
- níveis de isolamento.

Os testes deverão constituir **experiências reproduzíveis**, indicando:

- objetivo;
- estado inicial;
- configuração;
- comandos executados em cada sessão;
- ordem temporal das operações;
- resultado esperado;
- resultado observado;

- estado final;
- análise e conclusão.

Uma experiência poderá ser documentada, por exemplo, através de:

Passo	Sessão A	Sessão B	Resultado/observação
1	`BEGIN`		
2	`UPDATE ...`		
3		`BEGIN`	
4		`UPDATE ...`	...
5	`COMMIT`		...
6		...	...

Capturas de ecrã poderão ser utilizadas como evidência complementar, mas não substituem a documentação necessária para reproduzir a experiência.

7.7. Desempenho e índices

Deverá ser realizada uma experiência de avaliação de desempenho utilizando um volume de dados significativamente superior ao utilizado nos testes funcionais.

Poderão ser utilizados dados sintéticos gerados especificamente para este efeito.

Deverão ser utilizados:

```
EXPLAIN
```

e:

```
EXPLAIN ANALYZE
```

Deverá ser estudado pelo menos um caso relevante de utilização de **índices**, comparando uma ou mais consultas antes e depois da introdução do índice.

A análise deverá considerar não apenas os tempos de execução, mas também as alterações relevantes no **plano de execução** produzido pelo PostgreSQL.

A criação do índice deverá ser justificada pelas características dos dados e das consultas.

8. Ferramentas

Para modelação e documentação são recomendados:

- **diagrams.net/draw.io** — modelo conceptual ER, preferencialmente em notação de Chen;
- **Markdown** — documentação;
- **Visual Studio Code** ou outro editor adequado;
- **Git e GitHub** — versionamento, colaboração e entregas.

O SGBD utilizado será:

- **PostgreSQL**.

Para interação com PostgreSQL poderão ser utilizados:

- **pgAdmin**;
- `psql`;
- extensões adequadas para **Visual Studio Code**;
- outros clientes PostgreSQL adequados.

Os testes de transações e concorrência deverão utilizar sessões independentes de acesso ao SGBD.

9. Trabalho em grupo e responsabilidade individual

O projeto será realizado em grupos de **dois estudantes**.

Ambos os elementos são responsáveis pela totalidade do projeto e deverão compreender as decisões tomadas nas diferentes fases.

A distribuição de tarefas é natural, mas não deverá resultar numa divisão permanente em que um estudante realiza apenas a modelação e outro apenas a implementação ou os testes.

O histórico Git constitui uma fonte de informação sobre o processo de desenvolvimento, mas a contribuição individual não será avaliada apenas através do número de commits.

Será realizada uma **defesa/discussão individual/grupo**, durante a qual cada estudante deverá ser capaz de explicar e discutir qualquer componente do projeto.

10. Avaliação final

A avaliação quantitativa será realizada **apenas no final do semestre**.

A. Análise e modelação conceptual — 25%

Critério	Peso
Análise e delimitação do domínio	3%

Critério	Peso
Requisitos de dados	4%
Regras de negócio	4%
Atores e necessidades de acesso	3%
Necessidades de informação	3%
Modelo conceptual ER e respetiva coerência	8%
Total	**25%**

B. Projeto lógico e implementação — 35%

Critério	Peso
Modelo relacional e mapeamento a partir do modelo conceptual	11%
Análise e normalização	8%
Implementação PostgreSQL e integridade declarativa	13%
Qualidade e adequação dos dados	3%
Total	**35%**

C. Exploração e testes — 33%

Critério	Peso
Consultas SQL e vistas	8%
Papéis, privilégios e controlo de acesso	4%
Integridade, triggers e respetivos testes	4%
Transações	5%
Concorrência e isolamento	6%
Desempenho, índices e planos de execução	6%
Total	**33%**

D. Qualidade global e processo de desenvolvimento — 7%

Este critério será avaliado **transversalmente sobre as três fases**.

Serão considerados:

- qualidade e clareza da documentação;
- organização e legibilidade dos artefactos;
- coerência entre análise, modelos, implementação e testes;

- rastreabilidade entre requisitos, regras, necessidades de acesso e respetivas implementações;
- evolução entre `v1.0`, `v2.0` e `v3.0`;
- consideração adequada do feedback recebido;
- utilização adequada de Git/GitHub;
- participação dos dois elementos;
- reprodutibilidade da base de dados e das experiências.

11. Cronograma

Período	Trabalho principal	Marco
Semanas 1–5	Análise, requisitos, regras, atores, necessidades de informação e modelo conceptual	**Semana 5 — `**v1.0**` + feedback qualitativo**
Semanas 6–10	Modelo relacional, normalização, PostgreSQL, dados e início das consultas	**Semana 10 — `**v2.0**` + feedback qualitativo**
Semanas 11–fim	Consultas, vistas, controlo de acesso, triggers, transações, concorrência e desempenho	**Final — `**v3.0**` + avaliação quantitativa**

12. Princípio geral

O projeto deverá permitir acompanhar e justificar todo o percurso:

problema do mundo real
↓
requisitos, regras de negócio e atores
↓
modelo conceptual
↓
modelo relacional e normalização
↓
implementação PostgreSQL
↓
dados, consultas e vistas
↓
integridade e controlo de acesso
↓
transações e concorrência
↓
desempenho

↓

testes e validação

Deverá existir **rastreabilidade** entre as necessidades identificadas e a solução implementada.

Por exemplo:

regra de negócio → modelo → restrição/trigger → teste

necessidade de informação → consulta/view

ator → necessidade de acesso → ROLE → privilégio → teste

Os mecanismos SQL deverão resultar das necessidades reais do problema. Não deverão ser introduzidas consultas, vistas, triggers, roles ou índices apenas para demonstrar a sua utilização.

A qualidade das decisões, a sua justificação, a coerência entre as diferentes etapas e a capacidade de testar e reproduzir a solução constituem elementos fundamentais do projeto.